

Parkolóhely-foglalás - Backend házi feladat

1. Keretek

- **Nyelv/keretrendszer-független:** bármilyen backend stack-kel megoldható.
 - **Javasolt időkeret:** 3-4 óra aktív munka.
 - **Beadási határidő:** a kiadástól számított 48 óra — ez **nem ugyanaz**, mint a javasolt időkeret, nyugodtan oszd el több nap alatt, ha úgy kényelmesebb.
 - **Elvárás a megoldással szemben:** legyen **bug-mentes** és törekedj **jó teljesítményre** — nincs konkrét terhelési célszám megadva, de a rendszertervben indokold, milyen megfontolásokat tettél ezek érdekében.
 - **Az interfészt (API, CLI, vagy bármi más) te tervezed** — nincs előre adott végpont-lista vagy kontraktus.
 - **Adatbázis-használat kötelező** — pusztán memóriában tartott állapot nem elegendő. A konkrét technológiát (relációs, dokumentum-alapú, stb.) és a sémát te választod, de a rendszernek **induláskor inicializált, feltöltött állapotban** kell lennie (pl. alap referenciaadatokkal), hogy azonnal tesztelhető legyen.
 - **AI-eszközök (ChatGPT, Copilot, stb.) használata javasolt** — nyugodtan élj velük a megoldás során, ez nem hátrány. Amit mérünk, az nem az, hogy AI nélkül oldod-e meg, hanem hogy érted és tudod-e indokolni, amit beadsz (lásd 4. szakasz).
-

2. A feladat

Egy parkoló foglalási rendszerét kell megterveznie és megvalósítania. A rendszer:

- nyilvántartja a parkolóhelyeket,
- fogad foglalási kérést (parkolóhely, kezdő- és záró időpont, kérelmező),
- eldönti, elfogadható-e a kérés,
- lekérdezhetővé teszi egy adott parkolóhely foglalásait,
- lehetővé teszi egy foglalás lemondását.

Extra (opcionális, nem kötelező): a parkolóhelyek nem feltétlenül egyformák (pl. vannak korlátozott használatú helyek). Ha van rá időd, gondold végig, ez hogyan hat a foglalási logikára, és valósítsd meg.

Futtatási elvárás: a teljes rendszer (kóddal és adatbázissal együtt) egy paranccsal induljon (pl. `docker-compose up`).

3. Elvárt mellékletek

A forráskód mellett add be:

- **Rendszerterv** — hogyan épül fel a megoldásod, milyen főbb komponensekből áll.

- **API-leírás** — milyen műveleteket, milyen formában lehet elérni.
- **Felhasználói kézikönyv** — hogyan kell használni a rendszert.

Mindhárom dokumentum **saját, tetszőleges formában és mélységben** készülhet — nincs előre megadott sablon vagy elvárt terjedelem.

4. Döntési Napló + rövid reflexió — kötelező melléklet

A fejlesztés során minden bizonnyal olyan pontokra fogsz bukkanni, amiket a feladateleírás nem specifikál részletesen. Ezeket dokumentáld az alábbi táblázatban:

#	Döntési pont	Amit választottál	Miért	Milyen alternatívát vetettél el
1				
2				
3				

- Minimum 3-4 kitöltött sor.
- **Rövid összefoglaló (max 5-6 mondat):** milyen problémákba ütköztél a megoldás során, hogyan oldottad meg őket, és mi volt a legnagyobb kihívás?
- **AI-eszköz használat:** AI-asszisztens (ChatGPT, Copilot, stb.) használata javasolt a megoldás során. Írd le a reflexióban, mire és hogyan használtad. **A nyers prompt history (export) csatolása kötelező**, ha használtál AI-t — ez nem hátrány, hanem a folyamatod dokumentálásának része.

5. Beadási formátum

- Forráskód, egy paranccsal futtatható (adatbázissal együtt, előre feltöltött/inicializált állapotban) — **publikus GitHub repóban**
- Törekedj rá, hogy a commit-history MVP-szerűen épüljön fel (kis, működő lépésekben haladva, ne egyetlen bemásolt végállapot) — ez segít nyomon követni a gondolatmenetedet
- Saját tesztek (unit/integration) a repóban
- Rendszerterv + API-leírás + felhasználói kézikönyv, külön fájlban/fájlokban
- Döntési Napló + reflexió, külön fájlban
- AI prompt history (nyers export), ha volt AI-eszköz használat